Synthetic Data Fidelity & Validation Asset Library

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

- What this is . . . 4
- How it works . . . 5
- Target data model . . . 5
- Benefits . . . 5
- Limitations . . . 6
- Deployment model — Assess → Design → Generate/Build → Validate → Expand . . . 6

Requirements 6

- 1. A synthetic dataset under test . . . 7
- 2. A governed Reference Profile source (aggregate-only) . . . 7
- 3. A field manifest (schema typing) for the in-scope fields . . . 7
- 4. Fidelity/utility tolerances and coverage targets (the pass/fail gates) . . . 8
- 5. A utility evaluation task (TSTR-style), where utility is in scope . . . 8
- 6. A version-controlled asset store . . . 8

Global Requirements — Synthetic Data & AI Test-Enablement Solutions 9

- 1. A Python data/engineering environment . . . 9
- 2. A target clinical data model / standard . . . 9
- 3. An isolated non-production test environment . . . 9
- 4. Privacy, governance & compliance stakeholders and policy . . . 10
- 5. AI/QA use cases and a pilot scope . . . 10
- 6. Reference/real-data access for fidelity (read-only, minimized) . . . 10

Implementation Guide 11

- Stage 1 — Assess . . . 11
- Stage 2 — Design (author the reusable assets) . . . 11
- Stage 3 — Generate/Build (obtain the dataset under test) . . . 12
- Stage 4 — Validate (measure, gate, report) . . . 12
- Stage 5 — Expand (grow the library) . . . 12

Testing & Validation 13

- Test 1 — Reference Profile is aggregate-only and complete . . . 13
- Test 2 — Field manifest maps cleanly to the dataset . . . 13
- Test 3 — Statistical fidelity within tolerance . . . 13
- Test 4 — Utility (fit-for-use) within gap tolerance . . . 14
- Test 5 — Coverage & edge cases met . . . 14
- Test 6 — Privacy/disclosure signals reviewed . . . 14

Test 7 — Evidence Report renders an objective overall verdict . . . 14
Test 8 — Assets are reusable on a second dataset . . . 14
Troubleshooting 15
The Reference Profile build refuses to emit ("contains row-level data") . . . 15
Manifest fields not found in the synthetic dataset . . . 15
Fidelity passes but the model is worse on synthetic data (utility gap) . . . 15
Correlation preservation is low despite good per-field distributions . . . 15
A coverage rule reports RULE_ERROR . . . 15
Coverage fails because a required cohort is absent . . . 16
Privacy check reports a FAIL (exact match or nearest-neighbor) . . . 16
The runner reports utility "NOT_MEASURED" but you expected it measured . . . 16
`synfid_validate.py` cannot import the sibling tools . . . 16
Evidence Reports differ run-to-run on the same inputs . . . 16
Best Practices 16
Keep fidelity, utility, and privacy as three separate verdicts . . . 17
Utility is the acceptance test, not statistical fidelity . . . 17
Build the Reference Profile once, aggregate-only, in-place . . . 17
Parameterize what changes; freeze the method . . . 17
Set tolerances with the AI/QA owners, before measuring . . . 17
Coverage is spec-driven, not discovered . . . 17
Version every asset; grow the library deliberately . . . 17
Make failures actionable . . . 17
Re-validate when anything upstream changes . . . 17
Defer legal/regulatory calls explicitly . . . 18
Deliverables Summary 18
Documentation . . . 18
The 5 Python tools (`SYNFID-Scripts.zip`) . . . 18
The reusable assets you build with them . . . 19
Appendix A — PowerShell Scripts 20

Executive Summary

Outcome. Every synthetic dataset the organization ships is accompanied by machine-generated **evidence of quality** — proof that it is faithful enough, useful enough, and complete enough for its AI use case — and the validation itself becomes a **reusable asset library** instead of work repeated for each dataset.

The problem. Synthetic data is only valuable if it is *faithful enough* for the AI use case, and teams rarely have a defensible, repeatable way to prove it. Two silent failures recur: data that looks statistically fine but trains a worse model (a utility gap), and data that omits the rare-but-critical cohorts the model most needs (a coverage gap). Meanwhile each team re-invents validation from scratch.

The approach. This solution builds a **fidelity and validation asset library**. It measures **statistical fidelity** (distribution distances, category frequencies, correlation preservation) and **utility** (a Train-on-Synthetic, Test-on-Real gap) against a governed, aggregate-only Reference Profile; it checks **coverage and edge cases** against an explicit specification; and it reports **privacy/disclosure signals** for the privacy office. These are packaged as reusable assets — a Field Manifest, Reference Profile, metric suite, coverage spec, and a runnable Validation Suite that emits an **Evidence Report** with objective pass/fail gates. Five Python tools operationalize the workflow. Work follows a pilot-first **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model with explicit gates and rollback.

Why it is safe. No real PHI is written to a shared synthetic harness, twin, or AI test environment; reference access is read-only and minimized, and the exported Reference Profile is aggregate-only. Automation is Python. Synthetic-data and AI/model tools are referenced at the pattern level, with version-specific behavior deferred to the vendor's documentation. Critically, the library supplies the **technical method and evidence only** — HIPAA/HITRUST de-identification determinations, IRB/privacy sign-off, and disclosure-risk acceptance remain with the organization's privacy, compliance, and legal functions.

Bottom line. Fidelity and utility measured, coverage checked, and validation assets reused — so AI teams can trust the synthetic data, and the organization stops paying to re-prove quality one dataset at a time.

Solution Overview

What this is

The **Synthetic Data Fidelity & Validation Asset Library** turns "we think our synthetic data is good enough" into **evidence you can show, defend, and reuse**. It gives a data/platform engineering team a repeatable way to (1) **measure how faithfully** a synthetic dataset reproduces the statistical shape and analytic behavior of governed real reference data, (2) **check that coverage and edge cases** the AI use case depends on are actually present, and (3) **package the metrics, thresholds, and checks as reusable validation assets and test suites** — so every synthetic dataset that ships carries an **evidence report of quality**, and teams stop re-inventing validation for each new dataset.

Fidelity is treated as three distinct questions that must not be collapsed into one:

- **Statistical fidelity** — do the synthetic distributions, correlations, and category

frequencies match the reference within an agreed tolerance?

- **Utility (fit-for-use)** — does a model or analysis behave equivalently on synthetic data as

on real data for the target AI use case? (Fidelity without utility is a nice-looking dataset that trains a worse model.)

- **Coverage / edge cases** — are the rare-but-critical cohorts, value ranges, and combinations

the AI use case must handle actually represented?

Privacy/disclosure risk is a **separate axis** that this library reports on technically but does **not** adjudicate — see Limitations.

How it works

The library is organized around a small set of **named, reusable assets** that you build once and reuse across datasets:

- **Reference Profile** — a governed, aggregate-only statistical fingerprint of the real data

(per-field distributions, correlations, category frequencies, missingness), built from read-only, minimized reference access. It contains **no row-level real data**.

- **Fidelity Metric Suite** — a configured set of distribution-distance and

correlation-preservation metrics with pass/fail tolerances.

- **Coverage Specification** — the cohorts, value ranges, and combinations the AI use case

requires, expressed as checkable rules.

- **Validation Suite** — the metric suite + coverage spec + tolerances bound together as one

runnable test suite that emits an **Evidence Report**.

Five Python tools operationalize these assets (all safe-by-default, `--dry-run` where they write, standard-library-first). They are named and described in `08-deliverables-summary.md`.

Target data model

The library is **schema-driven and model-agnostic**: it works against whatever clinical data model the organization has standardized on — **FHIR**, **OMOP CDM**, or a source system's native schema (see global requirement §2). You supply a **field manifest** (which fields are numeric / categorical / datetime / identifier, and which are in scope) and the tools compute metrics per field and per relationship. This is why the assets are reusable: the *method* is fixed, the *field manifest and reference profile* change per dataset.

Benefits

- **Ship evidence, not assertions** — each synthetic dataset ships with a machine-generated

Evidence Report tying it to explicit pass/fail gates.

- **Stop re-inventing validation** — the metric suite, coverage spec, and thresholds are reused;

new datasets inherit a proven Validation Suite.

- **Catch the two silent failures** — a dataset that looks statistically fine but is useless for

the model (utility gap), or one that omits the rare cohort the model most needs (coverage gap).

- **Make trade-offs visible** — fidelity, utility, and privacy pull against each other; the

reports surface all three so decisions are informed, not accidental.

Limitations

- **Not a privacy/legal determination.** The library reports technical disclosure-risk signals (e.g. exact-match / nearest-neighbor indicators). **HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, and disclosure-risk acceptance are deferred to the organization's privacy, compliance, and legal functions.**
- **Only as good as the reference.** Fidelity is measured against the Reference Profile; if the reference statistics are unrepresentative, the metrics are too. The reference must be governed and agreed (global requirement §6).
- **No real PHI in a shared harness.** Reference access is read-only and minimized; the Reference Profile is aggregate-only. No real PHI is written to a shared harness, twin, or AI test environment.
- **Vendor-neutral at the pattern level.** Where a synthetic-data platform (e.g. Synthea, a commercial generator) or an AI/model framework is involved, it is referenced by capability; verify version-specific behavior against that vendor's documentation.

Deployment model — Assess → Design → Generate/Build → Validate → Expand

Every Synthetic solution follows the same pilot-first model with explicit pass/fail gates and a documented rollback:

1. **Assess** — confirm the AI use case, the target data model, the governed reference source, and the isolated environment. Decide which fidelity, utility, coverage, and privacy signals matter for *this* use case.
1. **Design** — author the **Reference Profile**, the **Fidelity Metric Suite** (with tolerances), and the **Coverage Specification** as the reusable assets. Define the pass/fail gates.
1. **Generate/Build** — obtain or generate the synthetic dataset under test (generation itself is handled by the org's synthetic-data tool; this library validates it).
1. **Validate** — run the **Validation Suite** on the pilot dataset; produce the **Evidence Report**; apply the pass/fail gates. Fail → rollback (do not release the dataset) and iterate.
1. **Expand** — once the pilot passes, promote the Validation Suite as a **reusable asset** and run it against subsequent datasets, growing the library.

Requirements

These are the **solution-specific** prerequisites for the Synthetic Data Fidelity & Validation Asset Library. They build on — and do **not** repeat — the six category-wide requirements in `global-requirements-synthetic.md` (Python environment, a target clinical data model, an isolated

non-production environment, privacy/governance stakeholders and policy, AI/QA use cases + pilot scope, and read-only minimized reference/real-data access). Confirm those six first; the items below are what *this* solution additionally needs.

> ***Boundary reminder.*** *No real PHI is written to a shared harness, twin, or AI test environment. Reference access is read-only and minimized. Privacy/de-identification determinations are deferred to the organization's privacy, compliance, and legal functions.*

1. A synthetic dataset under test

• **Description:** The synthetic dataset(s) to be validated, in a tabular form (CSV/Parquet) or an export from the org's synthetic-data platform, matching the in-scope fields of the target data model.

• **Why required:** The library *validates* synthetic data; generation is done by the org's tool. Without a dataset to score, there is nothing to measure.

• **How to verify:** Confirm the dataset exists in the isolated environment, is readable, and its columns map to the field manifest (below).

• **How to implement if missing:** Generate or export the synthetic dataset from the org's synthetic-data tool (e.g. Synthea or a commercial platform — see the vendor's documentation) into the isolated environment, then name it `Synthetic Dataset – <Use Case> – Pilot – v1`.

2. A governed Reference Profile source (aggregate-only)

• **Description:** Read-only, minimized access to **reference statistics** — either a governed real dataset (under the §4/§6 privacy controls) or agreed published reference distributions — from which an **aggregate-only** Reference Profile can be built. Row-level real data is never copied into the harness.

• **Why required:** Statistical fidelity and utility are measured against this reference; without it, fidelity claims are unfounded.

• **How to verify:** Confirm the reference source, its access terms, and that only aggregate statistics (not rows) will leave the governed environment.

• **How to implement if missing:** Agree the reference approach with the privacy office; build the **Reference Profile – <Use Case> – v1** in-place inside the governed environment and export only the aggregate profile.

3. A field manifest (schema typing) for the in-scope fields

• **Description:** A per-field declaration of **type** (numeric / categorical / datetime / identifier / free-text) and **role** (in scope for fidelity, key for correlation, cohort key for coverage, quasi-identifier for privacy signals), for the in-scope fields of the target model.

• **Why required:** Every metric is type-aware — distribution distance for numerics, frequency divergence for categoricals, correlation preservation across numeric pairs. Identifiers and free-text are excluded from statistical metrics. The manifest is what makes the assets reusable.

• **How to verify:** Confirm each in-scope field has a type and role; confirm identifiers/free-text are flagged out of statistical scope.

• **How to implement if missing:** Author the manifest with the AI/QA and data teams and store it as **Field Manifest – <Use Case> – v1** (the config helper tool scaffolds this).

4. Fidelity/utility tolerances and coverage targets (the pass/fail gates)

• **Description:** The agreed numeric **tolerances** (e.g. maximum acceptable distribution distance, minimum correlation-preservation, maximum utility gap) and **coverage targets** (which cohorts/ranges/combinations must be present and at what minimum representation) for this AI use case.

• **Why required:** "Faithful enough" is use-case-specific. Without agreed thresholds, the Evidence Report cannot render an objective pass/fail and the Validate gate is subjective.

• **How to verify:** Confirm the thresholds are documented and signed off by the AI/QA owners.

• **How to implement if missing:** Elicit thresholds with the AI/QA owners; record them in the Validation Suite config as **Validation Suite – <Use Case> – v1**.

5. A utility evaluation task (TSTR-style), where utility is in scope

• **Description:** A concrete, runnable **downstream task** representative of the AI use case (a classification/regression target, or an analytic query) used for **Train-on-Synthetic, Test-on-Real (TSTR)** — or, where real data cannot leave the governed environment, an equivalent utility comparison run inside it.

• **Why required:** Utility ("fit-for-use") cannot be inferred from statistical fidelity alone; it must be measured on a task that mirrors how the AI use case will consume the data.

• **How to verify:** Confirm a representative task, target variable/metric, and evaluation split are defined.

• **How to implement if missing:** Define the smallest representative task with the AI/QA team and record it as **Utility Task – <Use Case> – v1**.

6. A version-controlled asset store

• **Description:** A location (a repo or governed artifact store in the isolated environment) to hold the reusable assets — Field Manifest, Reference Profile (aggregate-only), Fidelity Metric Suite config, Coverage Specification, Validation Suite config, and Evidence Reports — under version control.

• **Why required:** Reuse and auditability depend on the assets being versioned and discoverable; this is what lets the library *grow* rather than be rebuilt per dataset.

• **How to verify:** Confirm the store exists, is access-controlled, and holds no row-level real data.

• **How to implement if missing:** Provision the store and commit the pilot assets as the first entries.

Global Requirements — Synthetic Data & AI Test-Enablement Solutions

These requirements apply to **every** solution in the **Synthetic** category (synthetic clinical data, hospital-twin simulation, and safe AI test enablement). Verify each before starting a solution's steps.

> ***Safety, vendor & regulatory boundary (applies to every Synthetic solution).*** *These solutions generate and validate* ***synthetic*** *(artificial) data and run in* ***isolated non-production*** *test environments.* ***Never place real Protected Health Information (PHI) into a shared synthetic harness, twin, or AI test environment*** *— where real data is touched at all it is read-only, minimized, and handled under the governing privacy controls.* ***Synthetic-data and digital-twin tools*** *(e.g. Synthea, and commercial platforms) and* ***AI/model frameworks*** *are integrated at the pattern/architectural level; specific vendor console internals are* ***cited or deferred*** *to that vendor's documentation, never invented.* ***Regulatory and legal interpretation*** *(HIPAA/HITRUST de-identification determinations, IRB/privacy-office sign-off, disclosure risk acceptance, and any ITAR/NERC/NRC or contract constraints) is* ***deferred to the organization's privacy, compliance, and legal functions*** *— the solutions provide the technical method and evidence, not legal determinations.*

1. A Python data/engineering environment

• **Description:** A supported **Python** environment (e.g. 3.10+) with the standard data toolchain (data frames, statistics, and the synthetic-data / validation libraries the solution uses).

• **Why required:** The automation and reusable assets are authored in Python (generation, de-identification, fidelity/utility metrics, scenario runners).

• **How to verify:** Confirm Python + the required packages install and run in the target environment.

• **How to implement if missing:** Provision the environment and install dependencies before starting.

2. A target clinical data model / standard

• **Description:** A defined **clinical data model or interoperability standard** to generate against — e.g. **FHIR**, **OMOP CDM**, or the source system's own schema — plus the fields/domains in scope (patients, encounters, conditions, medications, labs, vitals, notes, etc.).

• **Why required:** Realistic synthetic data and a credible hospital twin require a target schema and clinically plausible relationships; validation is measured against it.

• **How to verify:** Confirm the target model/standard and the in-scope domains with the AI/QA teams.

• **How to implement if missing:** Select and document the target model before generating.

3. An isolated non-production test environment

• **Description:** A **segregated, non-production** environment (compute + storage) for the synthetic harness, hospital twin, and AI test runs — separate from production and from real-PHI systems.

• **Why required:** Safe evaluation of AI solutions before broader adoption depends on isolation; it also bounds any residual disclosure risk.

• **How to verify:** Confirm the environment is isolated, access-controlled, and not connected to production PHI stores.

• **How to implement if missing:** Stand up / obtain the isolated environment before running.

4. Privacy, governance & compliance stakeholders and policy

• **Description:** The governing **privacy, data-governance, and compliance** policies and the named stakeholders (privacy officer, security, and — where real source data or de-identification claims are involved — IRB/legal) whose requirements the work must satisfy.

• **Why required:** Regulated healthcare data constraints, de-identification acceptance, and enterprise data governance are policy decisions the technical method must align to, not defaults.

• **How to verify:** Confirm the applicable policies, disclosure-risk threshold, and approvers.

• **How to implement if missing:** Identify the policies and stakeholders before generating or releasing test data.

5. AI/QA use cases and a pilot scope

• **Description:** The **AI use cases and QA scenarios** the synthetic data / twin must support, and a **pilot scope** to validate on before broader rollout.

• **Why required:** Fidelity, coverage, and scenario design are driven by the AI use cases; every solution follows an **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand** model proven on a pilot.

• **How to verify:** Confirm the target AI use cases, evaluation goals, and a named pilot.

• **How to implement if missing:** Elicit the use cases and define the pilot before building the harness.

6. Reference/real-data access for fidelity (read-only, minimized)

• **Description:** Where synthetic **fidelity/utility** must be measured against reality, **read-only, minimized** access to reference statistics or a governed real dataset (under the privacy controls in §4) — or agreed reference distributions where real data cannot be used.

• **Why required:** Fidelity and validation assets compare synthetic output to real-world distributions and relationships; without a reference the claims are unfounded.

• **How to verify:** Confirm the reference source and its access/governance terms.

• **How to implement if missing:** Agree the reference approach (governed real stats vs. published reference distributions) before validating.

*The deployment model across every Synthetic solution is the same: **Assess** → **Design** → **Generate/Build** → **Validate** → **Expand**, with explicit pass/fail (fidelity/privacy) gates and a documented rollback. Automation and reusable assets are delivered as **Python** — never real PHI is written to a shared harness, twin, or AI test environment.*

Implementation Guide

Follow these ordered steps to stand up the fidelity & validation asset library on a **pilot** AI use case, then promote the assets for reuse. The five Python tools referenced here are described in `08-deliverables-summary.md`; run each with `--help` first. Every step that produces an artifact **names** it with the `<Purpose> – <Scope> – <Stage>` convention and reuses that exact name later.

> ***Safety gates are load-bearing.*** *Do not release a synthetic dataset that fails the Validate gates. No real PHI is written to a shared harness/twin/AI test environment; reference access is read-only and minimized; the Reference Profile exported from the governed environment is **aggregate-only**.*

Stage 1 — Assess

1. **Confirm the six global requirements** (`global-requirements-synthetic.md`) and the six

solution-specific requirements (`02-requirements.md`) are met for the pilot use case.

1. **Fix the pilot scope.** With the AI/QA owners, name the pilot: e.g.

Fidelity Pilot – Readmission Model – v1. Record the AI use case, target data model (FHIR / OMOP / native), and the in-scope domains.

1. **Decide which signals matter** for this use case: which fields need statistical fidelity,

which correlations must be preserved, whether utility (TSTR) is in scope, which cohorts drive coverage, and which privacy signals to report. Record these as the scope for the Validation Suite.

Stage 2 — Design (author the reusable assets)

1. **Scaffold the config** with the config helper tool (`synfid_config.py`). Produce

Validation Suite – Readmission Model – v1 (a JSON config) that references the field manifest, reference profile, metric tolerances, coverage targets, and (if in scope) the utility task. Start from the template and edit values.

1. **Author the Field Manifest.** Populate **Field Manifest – Readmission Model – v1** — type and

role for every in-scope field; flag identifiers/free-text out of statistical scope. The config helper can infer a first draft from the synthetic dataset's columns for you to correct.

1. **Build the Reference Profile (aggregate-only).** Inside the governed environment, run the

reference-profile builder (`synfid_profile.py`) against the read-only, minimized reference to produce **Reference Profile – Readmission Model – v1** — per-field distributions, category frequencies, correlations, and missingness. **Export only this aggregate profile** from the governed environment; no rows leave.

1. **Author the Coverage Specification.** In the Validation Suite config, express the required

cohorts, value ranges, and combinations as checkable rules with minimum representation, e.g. `age>=80 AND readmitted=1` must be present at ≥ target. Name it **Coverage Spec – Readmission Model – v1**.

1. **Set the pass/fail gates.** Record the agreed tolerances in the config: max distribution

distance, min correlation-preservation, max utility gap, and coverage completeness. These are the objective gates the Validate stage enforces.

Stage 3 — Generate/Build (obtain the dataset under test)

1. **Obtain the synthetic dataset** from the org's synthetic-data tool into the isolated

environment and name it **Synthetic Dataset – Readmission Model – Pilot – v1**. (Generation is the org's tool; this library validates the output. Where a scenario/perturbation run is needed to exercise coverage, use the scenario/coverage runner — see step 12.)

1. **Sanity-check the dataset** loads and its columns map to **Field Manifest – Readmission

Model – v1** (the config helper validates this mapping).

Stage 4 — Validate (measure, gate, report)

1. **Measure statistical fidelity & utility.** Run the fidelity/utility metric suite

(`synfid_fidelity.py`) with the Validation Suite config against **Synthetic Dataset – Readmission Model – Pilot – v1** and **Reference Profile – Readmission Model – v1**. It computes per-field distribution distances, correlation-preservation, category-frequency divergence, and (if in scope) the TSTR utility gap.

1. **Check coverage & edge cases.** Run the coverage/edge-case checker

(`synfid_coverage.py`) against the dataset and **Coverage Spec – Readmission Model – v1**; it reports which required cohorts/ranges/combinations are present and at what representation.

1. **Run privacy/validation signals.** Run the privacy/validation check

(`synfid_privacy.py`) to report technical disclosure-risk signals (exact-match / nearest-neighbor indicators) against the reference profile. **These are signals for the privacy office, not a determination.**

1. **Produce the Evidence Report.** Run the validation-suite runner (`synfid_validate.py`) — it

orchestrates steps 11–13 under the Validation Suite config and emits **Evidence Report – Readmission Model – Pilot – v1** (machine-readable JSON + a human-readable summary) with an overall **PASS/FAIL** against every gate.

1. **Apply the gates (pass/fail).**

• **PASS** (all gates met) → the dataset is cleared for the pilot AI use case; attach the

Evidence Report to the dataset.

• **FAIL** (any gate breached) → **rollback**: do **not** release the dataset. Record the

failing gate(s), return to the org's generator to adjust generation (or to the coverage runner to add missing cohorts), and re-run Validate. Iterate until PASS.

Stage 5 — Expand (grow the library)

1. **Promote the assets.** Commit **Field Manifest / Reference Profile / Validation Suite – Readmission Model – v1 **to the version-controlled asset store as** reusable assets**.

1. **Reuse on the next dataset.** For a new synthetic dataset in the same domain, reuse the same Validation Suite (adjusting only what genuinely differs) and run steps 9–15 — new datasets now inherit a proven validation path and ship with their own Evidence Report.

1. **Track the library.** Maintain an index of assets and Evidence Reports so every shipped synthetic dataset is traceable to the gates it passed.

Testing & Validation

These procedures verify that the library itself works and that a synthetic dataset meets its gates. Each has an explicit **expected pass/fail** signal. Run them in the isolated environment; no real PHI is written there — reference inputs are aggregate-only.

Test 1 — Reference Profile is aggregate-only and complete

- **Procedure:** Build **Reference Profile – <Use Case> – v1** with `synfid_profile.py`, then inspect the output.
- **Expected PASS:** The profile contains per-field statistics (distributions, category frequencies, correlations, missingness) for every in-scope field and **no row-level real values**. File carries a manifest hash.
- **Expected FAIL:** Any in-scope field missing, or any row-level real data present → do not export; fix scope/config and rebuild.

Test 2 — Field manifest maps cleanly to the dataset

- **Procedure:** Run the config helper's validation on **Field Manifest – <Use Case> – v1** against **Synthetic Dataset – <Use Case> – Pilot – v1**.
- **Expected PASS:** Every manifest field exists in the dataset with a compatible type; identifiers/free-text flagged out of statistical scope.
- **Expected FAIL:** Unmapped columns or type mismatches → correct the manifest before measuring.

Test 3 — Statistical fidelity within tolerance

- **Procedure:** Run `synfid_fidelity.py` with the Validation Suite config.
- **Expected PASS:** Per-field distribution distances ≤ the configured max; category-frequency divergence ≤ max; correlation-preservation ≥ the configured min — for all gated fields/pairs.
- **Expected FAIL:** Any gated field/pair breaches its tolerance → the dataset fails the fidelity gate; return to generation.

Test 4 — Utility (fit-for-use) within gap tolerance

• **Procedure:** Where utility is in scope, run the TSTR-style utility comparison via `synfid_fidelity.py` (or the runner) using **Utility Task – <Use Case> – v1**.

• **Expected PASS:** The utility gap (real-trained vs synthetic-trained performance on the same real test measure) ≤ the configured max gap.

• **Expected FAIL:** Utility gap exceeds tolerance even if statistical fidelity passed — a silent failure this test exists to catch; the dataset fails the utility gate.

Test 5 — Coverage & edge cases met

• **Procedure:** Run `synfid_coverage.py` against **Coverage Spec – <Use Case> – v1**.

• **Expected PASS:** Every required cohort/range/combination is present at or above its minimum representation.

• **Expected FAIL:** Any required cohort missing or below minimum → coverage gate fails; add the missing cohorts (regenerate or run the scenario/coverage runner) and re-check.

Test 6 — Privacy/disclosure signals reviewed

• **Procedure:** Run `synfid_privacy.py`; route the output to the privacy office.

• **Expected PASS (technical):** Exact-match and nearest-neighbor indicators are within the agreed technical threshold **and** the privacy office has reviewed the signals.

• **Expected FAIL:** Indicators exceed the agreed technical threshold → escalate; **the determination remains with privacy/compliance/legal**, not the tool.

Test 7 — Evidence Report renders an objective overall verdict

• **Procedure:** Run `synfid_validate.py` to produce **Evidence Report – <Use Case> – Pilot – v1**.

• **Expected PASS:** The report aggregates Tests 3–6 into an overall **PASS** only when every gate passes, and is reproducible (same inputs → same verdict).

• **Expected FAIL:** Any gate FAIL yields overall **FAIL**; the report names the failing gate(s).

Test 8 — Assets are reusable on a second dataset

• **Procedure:** Point the same Validation Suite at a second synthetic dataset in the same domain and run the runner without re-authoring the assets.

• **Expected PASS:** The suite runs end-to-end and emits a new Evidence Report using the reused assets.

• **Expected FAIL:** The suite requires re-authoring to run → the assets are not truly reusable;

parameterize what changed (dataset path, dates) out of the fixed method.

Troubleshooting

Common failure modes when building and running the fidelity & validation asset library, with resolutions. All work happens in the isolated environment; no real PHI is written to a shared harness.

The Reference Profile build refuses to emit ("contains row-level data")

- **Cause:** The profile builder's safety check found row-level values in the output — usually a

field mis-typed so raw values were retained, or a manifest role not set.

- **Resolution:** Correct **Field Manifest – <Use Case> – v1** so identifiers/free-text are flagged

out of statistical scope, then rebuild. The profile must be aggregate-only before it leaves the governed environment.

Manifest fields not found in the synthetic dataset

- **Cause:** Column names differ (case, prefixes, model mapping) between the reference and the

synthetic dataset.

- **Resolution:** Run `synfid_config.py validate --config … --data …` to list the exact mismatches;

align the manifest field names to the dataset columns (or rename dataset columns on export).

Fidelity passes but the model is worse on synthetic data (utility gap)

- **Cause:** Statistical distances can look fine while the joint structure the model relies on is

distorted — the classic silent failure.

- **Resolution:** Ensure utility is **in scope** and measured: define **Utility Task – <Use Case> –

v1**, run TSTR (`--real-test-metric` / `--synth-test-metric`), and gate on `max_utility_gap`. If the gap fails, return to generation — do not release on statistical fidelity alone.

Correlation preservation is low despite good per-field distributions

- **Cause:** The generator reproduced marginals but not joint relationships between numeric fields.
- **Resolution:** This is expected behavior of the metric — it exists to catch exactly this. Report

the failing pairs to the generation team and regenerate with correlation-aware settings (see the generator vendor's documentation); re-run the fidelity suite.

A coverage rule reports RULE_ERROR

- **Cause:** The rule uses unsupported syntax. The checker supports a small, safe grammar:

`field<op>value` predicates joined by `AND`, ops `== != >= <= > <` (no `OR`, no functions, no `eval`).

- **Resolution:** Rewrite the rule within the grammar (split an `OR` into two coverage targets).

Quote string values or leave them bare; numerics compare numerically automatically.

Coverage fails because a required cohort is absent

- **Cause:** The generator under-produced a rare cohort the AI use case needs.
- **Resolution:** Add the cohort — regenerate with the generator's rare-case controls, or use the scenario/coverage runner to enumerate the needed slices — then re-check against **Coverage Spec – <Use Case> – v1**.

Privacy check reports a FAIL (exact match or nearest-neighbor)

- **Cause:** Synthetic quasi-identifier tuples too closely reproduce the reference sample.
- **Resolution:** Treat as a **signal**, not a verdict: escalate to the privacy office. Technically, regenerate with stronger privacy settings and/or reduce quasi-identifier granularity. **The de-identification determination remains with privacy/compliance/legal — the tool never makes it.**

The runner reports utility "NOT_MEASURED" but you expected it measured

- **Cause:** `utility_task.in_scope` is `true` in the config but no `--real-test-metric` / `--synth-test-metric` were supplied.
- **Resolution:** Supply both metrics (or set `in_scope` to `false` if utility genuinely isn't part of this suite). The runner deliberately fails rather than silently skip an in-scope gate.

`synfid_validate.py` cannot import the sibling tools

- **Cause:** The scripts were separated from each other; the runner imports `synfid_fidelity`, `synfid_coverage`, and `synfid_privacy`.
- **Resolution:** Keep all five scripts in the same directory (they ship together in `SYNFID-Scripts.zip`). The runner adds its own directory to `sys.path`; do not rename the modules.

Evidence Reports differ run-to-run on the same inputs

- **Cause:** Non-determinism upstream (a re-sampled reference, a changed dataset, or an unpinned generator), not the metrics — the metric code is deterministic for fixed inputs.
- **Resolution:** Pin the inputs: use the hashed **Reference Profile**, a fixed synthetic dataset, and a versioned config. Same inputs must yield the same verdict; if not, an input changed.

Best Practices

Durable practices for building and growing a fidelity & validation asset library.

Keep fidelity, utility, and privacy as three separate verdicts

Never collapse them into one number. A dataset can pass statistical fidelity and still fail utility (a worse model) or coverage (missing the rare cohort), and can be high-fidelity yet high-disclosure-risk. Report and gate each axis independently.

Utility is the acceptance test, not statistical fidelity

Statistical distances are necessary but not sufficient. Where an AI use case exists, make a TSTR-style utility gap the primary acceptance gate — it measures the thing you actually care about: does the model behave the same on synthetic data.

Build the Reference Profile once, aggregate-only, in-place

Compute the profile inside the governed environment and export only aggregate statistics. Never copy row-level real data into the harness. Hash the profile so every Evidence Report is traceable to the exact reference it was measured against.

Parameterize what changes; freeze the method

Reusability comes from separating the fixed *method* (metrics, tolerances, coverage rules) from the per-dataset *parameters* (dataset path, dates, reference profile). If validating a new dataset requires editing the metric code, refactor the difference into config.

Set tolerances with the AI/QA owners, before measuring

Agree "faithful enough" thresholds up front and record them in the Validation Suite config. Thresholds chosen after seeing the results are not gates — they are rationalizations.

Coverage is spec-driven, not discovered

Enumerate the required cohorts/edge cases from the AI use case and check the dataset against that spec. Do not infer coverage from whatever the generator happened to produce.

Version every asset; grow the library deliberately

Commit the Field Manifest, Reference Profile, metric suite, coverage spec, Validation Suite, and every Evidence Report to a version-controlled store. Each shipped dataset should be traceable to the gates it passed and the asset versions used.

Make failures actionable

A FAIL should name the specific gate and field/cohort so the generation team knows exactly what to fix. Route privacy signals to the privacy office as signals — never let a tool imply a de-identification determination.

Re-validate when anything upstream changes

A new generator version, a schema change, a shifted reference, or a new AI use case invalidates prior evidence. Re-run the suite and issue a fresh Evidence Report; do not rely on stale proof.

Defer legal/regulatory calls explicitly

HIPAA/HITRUST de-identification, IRB/privacy sign-off, and disclosure-risk acceptance belong to privacy/compliance/legal. The library supplies the technical method and evidence; keep that boundary visible in every report.

Deliverables Summary

What the buyer receives — a self-guided package to measure synthetic-data fidelity and build a reusable validation asset library, safe by default (no real PHI written to a shared harness; reference inputs are aggregate-only; privacy determinations deferred to the org).

Documentation

- **Solution Overview** — the fidelity model (statistical fidelity vs utility vs privacy),

coverage/edge cases, the reusable-asset approach, and the Assess → Design → Generate/Build → Validate → Expand deployment model.

- **Requirements Guide** — the six solution-specific prerequisites (synthetic dataset under test,

governed aggregate-only Reference Profile source, field manifest, tolerances/coverage targets, utility task, versioned asset store), each with how to verify and implement, on top of the six category-wide requirements.

- **Implementation Guide** — exact ordered steps to build the assets, measure a pilot dataset,

apply pass/fail gates, and promote the suite for reuse; every artifact named consistently.

- **Testing & Validation** — eight procedures (aggregate-only profile, manifest mapping, fidelity,

utility, coverage, privacy signals, evidence report, reusability) each with pass/fail.

- **Troubleshooting** — common failure modes and resolutions.
- **Best Practices** — durable practices for a growing validation library.
- **Executive Summary** — the outcome, approach, and why it is safe.

The 5 Python tools (`SYNFID-Scripts.zip`)

All are Python 3.10+, standard-library-first, argparse `--help`, safe-by-default (`--dry-run` where they write), no hardcoded secrets, no network calls, no real PHI.

1. `synfid_profile.py` **— Reference Profile builder.** Computes an **aggregate-only**

statistical fingerprint (per-field distributions, category frequencies, correlations, missingness) from a read-only, minimized reference, producing the reusable **Reference Profile**. Emits no row-level data.

1. `synfid_coverage.py` **— Coverage / edge-case checker & scenario runner.** Checks a synthetic

dataset against a **Coverage Specification** (required cohorts, value ranges, combinations, and minimum representation) and reports gaps; can enumerate the scenario slices the dataset must exercise.

1. `synfid_fidelity.py` **— Fidelity & utility metric suite.** Computes distribution distances,

category-frequency divergence, and correlation-preservation (synthetic vs Reference Profile), plus an optional **TSTR-style utility gap** — the core "is it faithful/useful enough" report.

1. `synfid_privacy.py` **— Privacy / disclosure-signal check.** Reports technical

disclosure-risk signals (exact-match and nearest-neighbor indicators) as inputs for the privacy office. Reports signals only — **never a de-identification determination**.

1. `synfid_config.py` **— Scope & config helper.** Scaffolds and validates the **Validation

Suite **config, drafts a** Field Manifest** from a dataset's columns, and checks manifest ↔ dataset mapping — the tool that makes the assets reusable.

The reusable assets you build with them

Field Manifest, Reference Profile (aggregate-only), Fidelity Metric Suite config, Coverage Specification, Validation Suite, and per-dataset **Evidence Reports** — versioned in your asset store so every synthetic dataset ships with evidence of quality and no team re-invents validation.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

synfid_config.py

synfid_coverage.py

synfid_fidelity.py

synfid_privacy.py

synfid_profile.py

synfid_validate.py